GSFC UNIVERSITY
EDUCATION RE-ENVISIONED

# Instance based learning and Bayesian learning

Dr. Anjali Diwan & Ms.Shweta Rajput

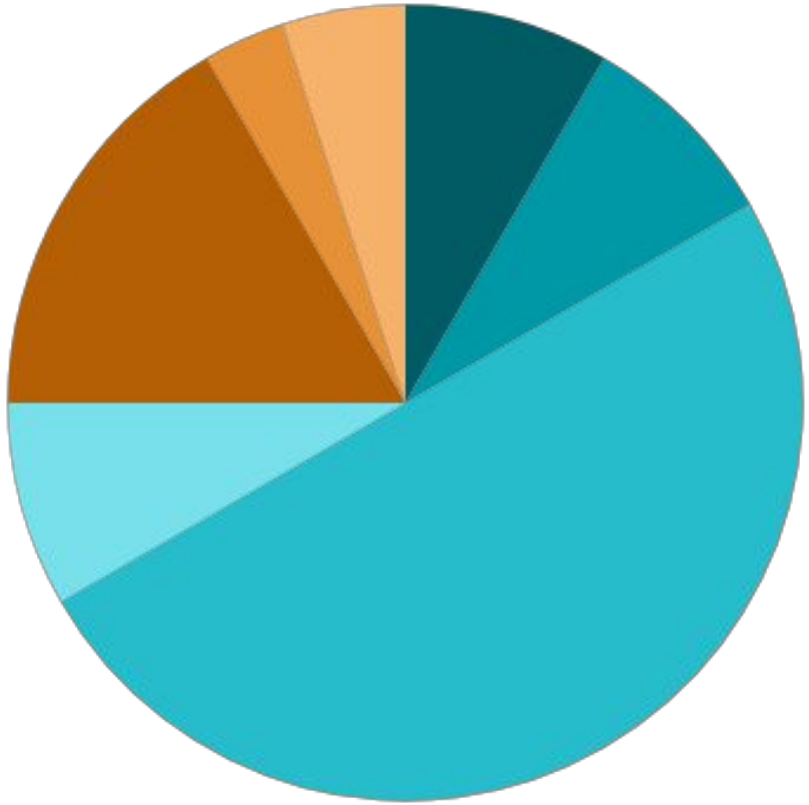
Department of Computer Science & Engineering

# Outline

1. Overview of basic concepts of ml
2. Instance based learning:
   - k- nearest neighbor learning
   - locally weighted regression
   - Radial basis & function
   - Case-based reasoning
3. Bayesian learning:
   - Bayes theorem and concept learning
   - Maximum likelihood and least- squared error hypothesis
   - Naïve bayes classifier
   - Bayesian belief Networks

*Source of Image: https://journey.prog-8.com/en/scenes/web-application-development/skills/object-oriented-programming/*

# Lecture Structure



| | |
|---|---|
| 05 Minutes | Outline & Objectives |
| 05 Minutes | Recap of Previous Lecture |
| 30 Minutes | Delivery of Actual Content as per Lesson Plan |
| 05 Minutes | Industry Relevance |
| 10 Minutes | Interactive Activities |
| 02 Minutes | Importance of Discipline |
| 03 Minutes | Recording of Attendance |

*Reference: Office Order No. Provost/092023135, Date: 18th September 2023*

# Objectives

1. Syllabus Discussion
2. Introduction & Background
3. Understand the basic concept of machine learning.
4. Understand the basic skills to decide which learning algorithm to use for what problem.
5. Able code up your own learning algorithm and evaluate and debug it.

# Syllabus Discussion

Syllabus Link

# Recap of Basics of Machine Learning

- Semester 4 FUNDAMENTALS OF AI & ML(BTCS405)
  - Introduction to ML - Supervised Learning & Optimization Techniques
    - Idea of Machine Learning from data, Supervised Learning, Linear and multi-class classifier, Linear and logistic regression, Decision Boundary, Cost Function Optimization, Introduction to Genetic Algorithm
  - Unsupervised Learning & Cluster Analysis
    - Unsupervised Learning, Clustering, K-mean clustering, hierarchical clustering, DBSCAN clustering, K-medoids clustering, Spectral clustering

# What is Machine Learning?

- Machine Learning is the science (and art) of programming computers so they can learn from data.
- Here is a slightly more general definition:
    - [Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed.

      —Arthur Samuel, 1959
- And a more engineering-oriented one:
    - A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.

      —Tom Mitchell, 1997

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Real World Example


In this case, the task T is to flag spam for new emails, the experience E is the training data, and the performance measure P needs to be defined; for example, you can use the ratio of correctly classified emails. This particular performance measure is called accuracy and it is often used in classification tasks.

# Why use Machine Learning?

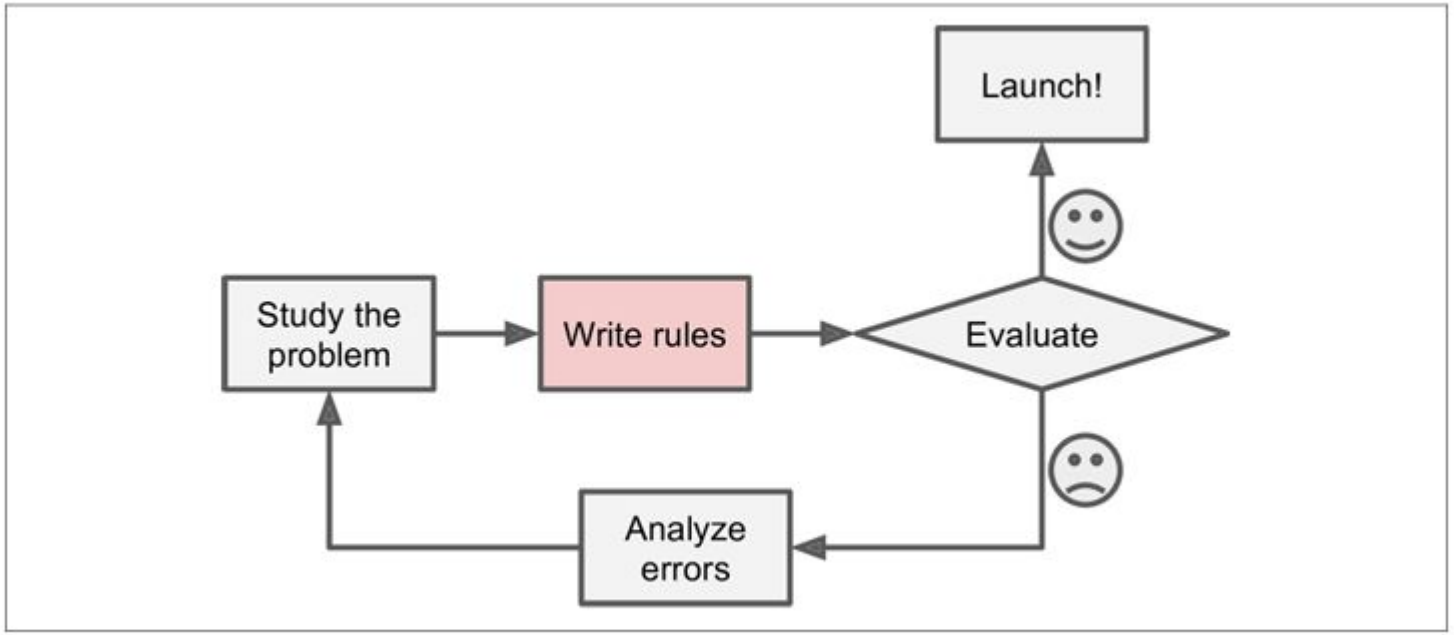



*Figure 1-1. The traditional approach*

- Analyze spam manually and **identify patterns** (e.g., "4U", "credit card", "free").
- **Write rules** or detection logic for each pattern.
- Test and **iterate manually** to improve performance.
- Results in a **complex, rule-based system** that is hard to maintain.
- **Requires constant updates** if spammers modify words (e.g., "4U" → "For U").

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Why use Machine Learning?

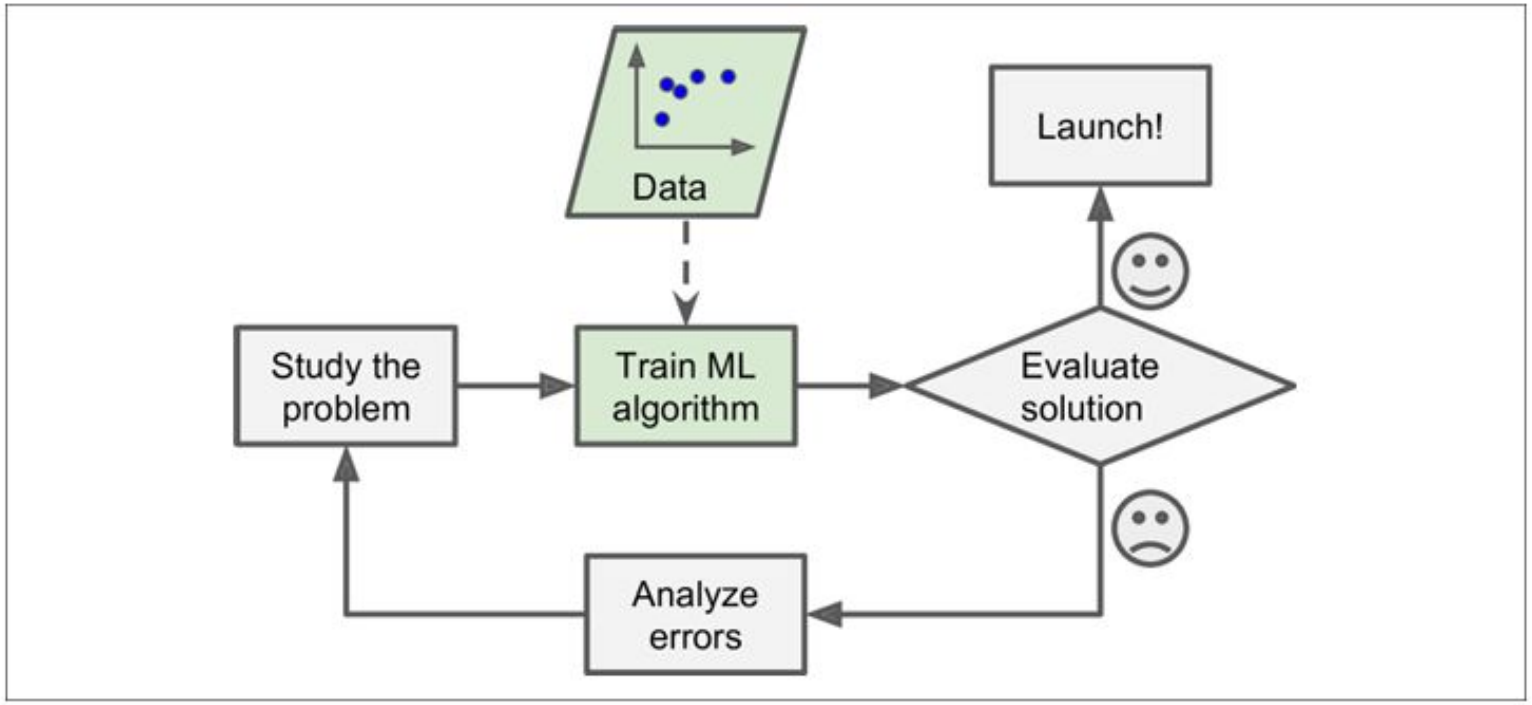



*Figure 1-2. Machine Learning approach*

- Learns patterns automatically from spam vs. ham examples.
- Detects frequent keywords and features statistically associated with spam.
- Shorter and easier-to-maintain code.

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Why use Machine Learning?

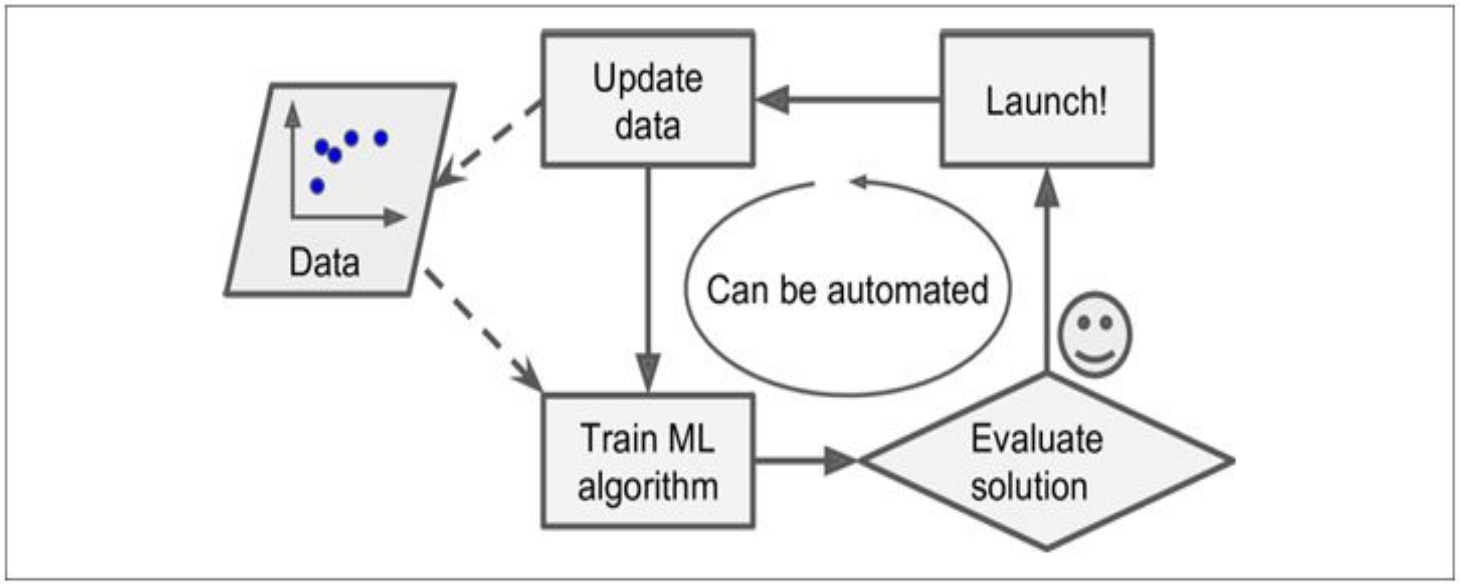



*Figure 1-3. Automatically adapting to change*

- Automatically adapts to new patterns like "For U" without manual intervention.
- Improves over time as more user-flagged spam is used for training.

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# What is Machine Learning?

- To summarize, Machine Learning is great for:
  - Problems for which existing solutions require a lot of hand-tuning or long lists of rules: one Machine Learning algorithm can often simplify code and perform better.
  - Complex problems for which there is no good solution at all using a traditional approach: the best Machine Learning techniques can find a solution.
  - Fluctuating environments: a Machine Learning system can adapt to new data.
  - Getting insights about complex problems and large amounts of data

- Types of Machine Learning:
  - Supervised Learning: Learning from labeled data.
  - Unsupervised Learning: Finding patterns in unlabeled data.
  - Reinforcement Learning: Learning via rewards and penalties.

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Types of Machine Learning

- **Whether or not** they are **trained** with **human supervision** (supervised, unsupervised, semi-supervised, and Reinforcement Learning)
- Whether or not they can **learn incrementally on the fly** (online versus batch learning)
- Whether they **work by simply comparing new data points to known data points**,or instead **detect patterns in the training data** and **build a predictive model**, much like scientists do (instance-based versus model-based learning)

*Image Source : https://images.app.goo.gl/EEpwoXqWr1gL1KRE7*

# Types of Machine Learning

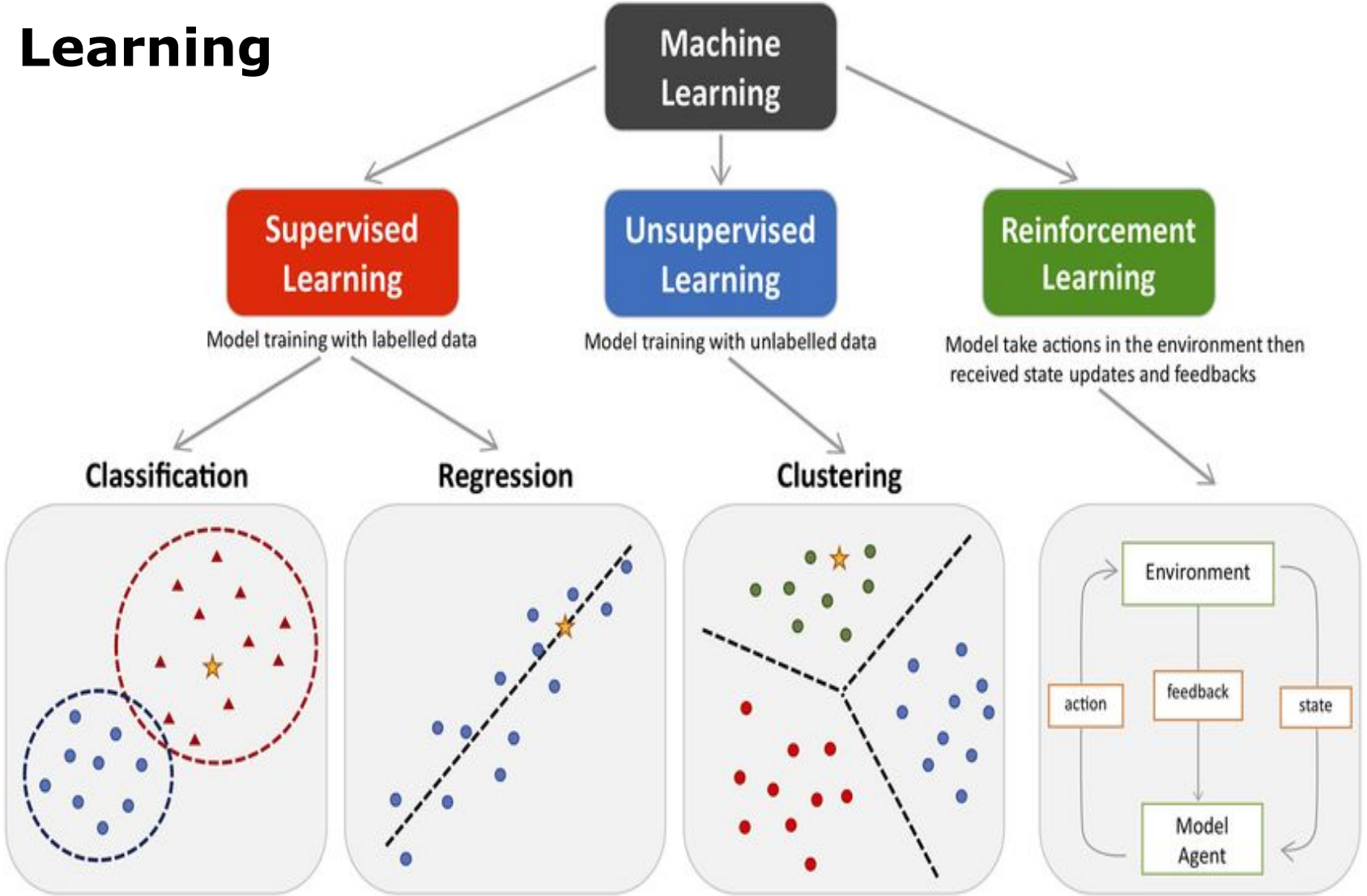



*Image Source : https://images.app.goo.gl/EEpwoXqWr1gL1KRE7*

# Supervised learning - Real world Example

- A typical supervised learning task is classification.
- Example
  - **The spam filter** : it is trained with many example emails along with their class (spam or ham), and it must learn how to classify new emails.
  - **Car Price Predictor** : task is to predict a target numeric value, such as the price of a car, given a set of features (mileage, age, brand, etc.) called predictors. This sort of task is called **regression**

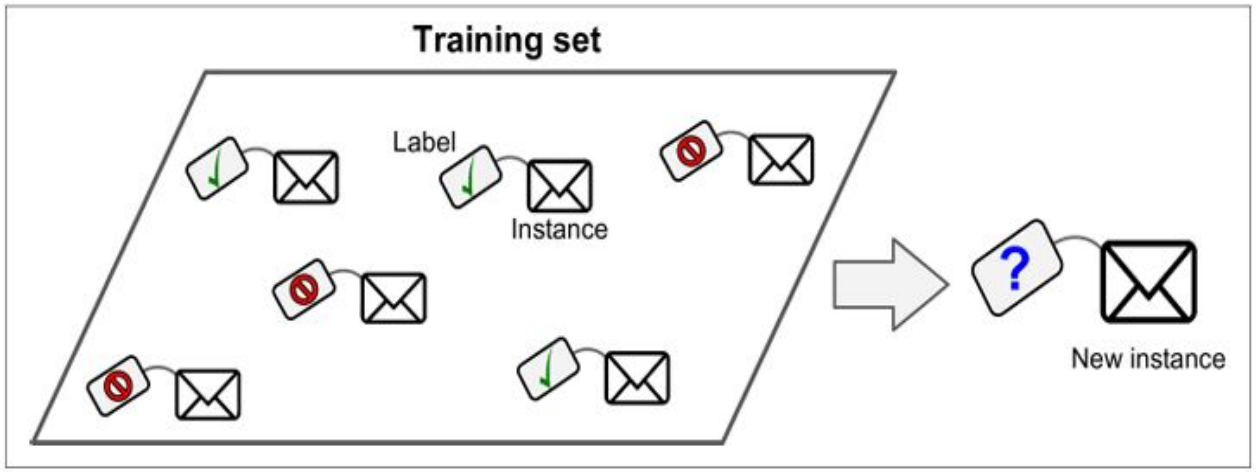



*Figure 1-5. A labeled training set for supervised learning (e.g., spam classification)*

**Mathematical Setup**

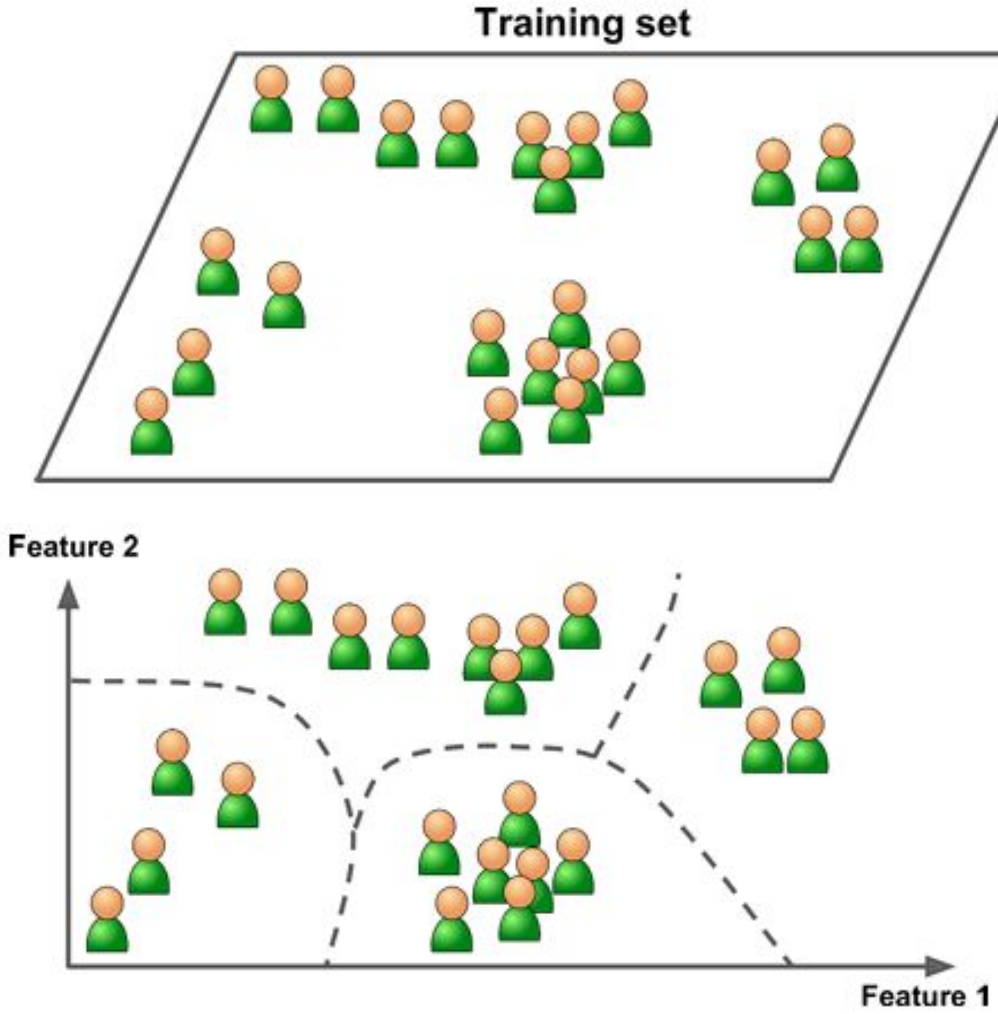
- **Input features:** $x_i \in \mathbb{R}^n$
- **Output label:** $y_i \in \mathbb{R}$ (for regression) or $y_i \in \{0, 1\}$ (for classification)
- **Dataset:**

$$D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)\}$$

- **Goal:** Learn a function $f : \mathbb{R}^n \to Y$ that maps $x_i$ to $y_i$, minimizing error.

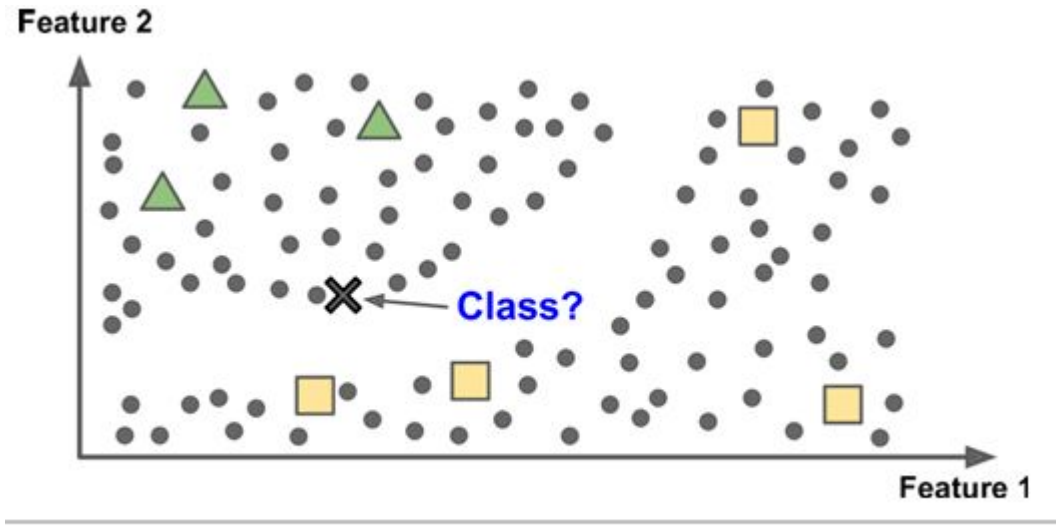
# Unsupervised learning - Real world Example

- In unsupervised learning, as you might guess, the training data is unlabeled. The system tries to learn without a teacher
- Example
  - **To detect groups of similar visitors** : it might notice that 40% of your visitors are males who love comic books and generally read your blog in the evening, while 20% are young sci-fi lovers who visit during the weekends, and so on. If you use a **hierarchical clustering algorithm**, it may also subdivide each group into smaller groups. This may help you target your posts for each group.

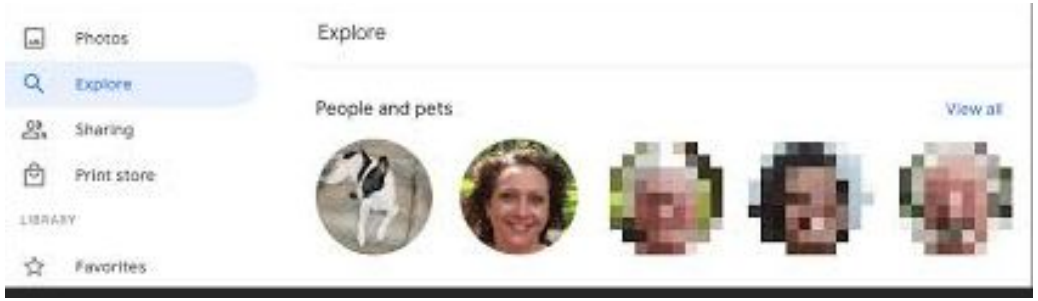


# Semi Supervised learning - Real world Example

- Partially labeled training data, usually a lot of unlabeled data and a little bit of labeled data
- Example
  - **Google Photos** : Once you upload all your family photos to the service, it automatically recognizes that the same person A shows up in photos 1, 5, and 11, while another person B shows up in photos 2, 5, and 7. This is the unsupervised part of the algorithm (clustering). Now all the system needs is for you to tell it who these people are. Just one label per person, 4 and it is able to name everyone in every photo, which is useful for searching photos

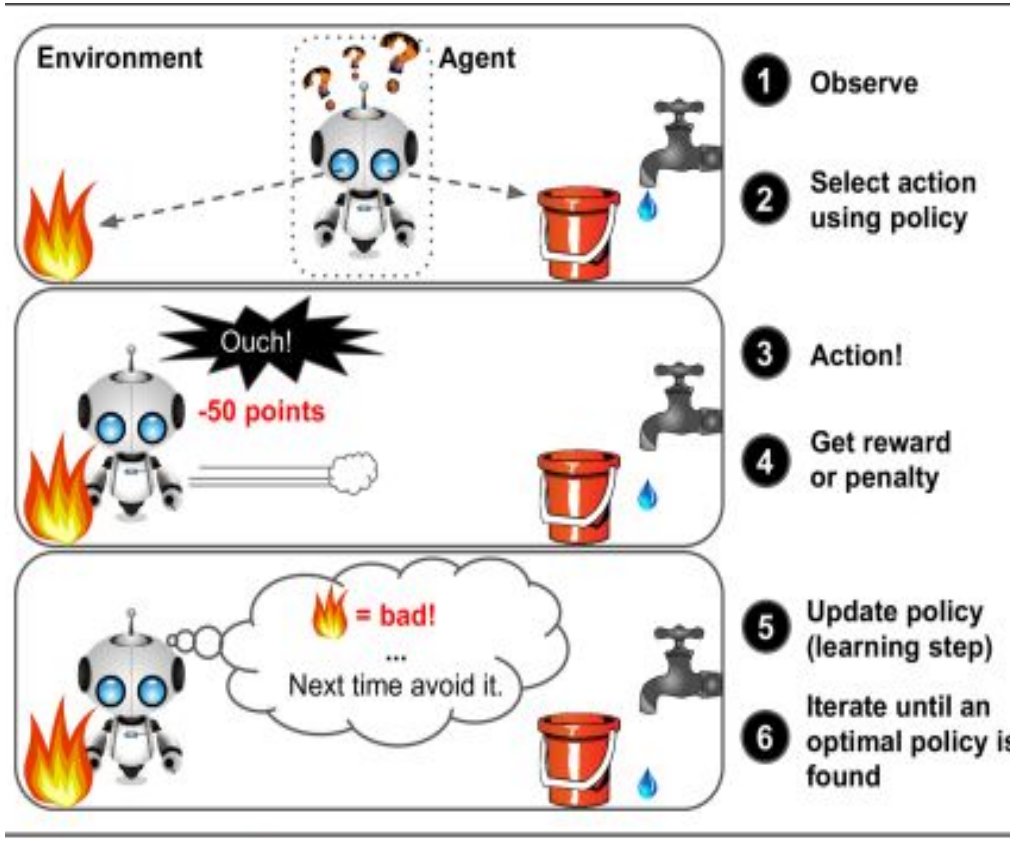



*Semisupervised learning*



# Reinforcement Learning - Real world Example

- The learning system, called an agent can observe the environment, select and perform actions, and get rewards in return or penalties in the form of negative rewards.
- It must then learn by itself what is the best strategy, called a policy, to get the most reward over time. A policy defines what action the agent should choose when it is in a given situation.
- Example **DeepMind's AlphaGo program** : In May 2017 beat the world champion Ke Jie at the game of Go. It learned its winning policy by analyzing millions of games, and then playing many games against itself. Learning was turned off during the games against the champion; AlphaGo was just applying the policy it had learned.



# Why ML for Intelligent Systems?

- Intelligent systems like recommendation engines, self-driving cars, and smart assistants rely on learning from interactions and data.
- ML provides the algorithms that drive adaptive, intelligent behavior in such systems.

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Instance-Based Learning (IBL)

- Instance-based learning, also called memory-based learning, is a lazy learning approach that stores all training instances and makes decisions only when queried (i.e., at prediction time).
    - It does not build an explicit general model.
    - Instead, it uses specific instances to predict new cases.
    - Emphasizes local approximation over global generalization.

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Key Concepts in Instance-Based Learning (IBL)

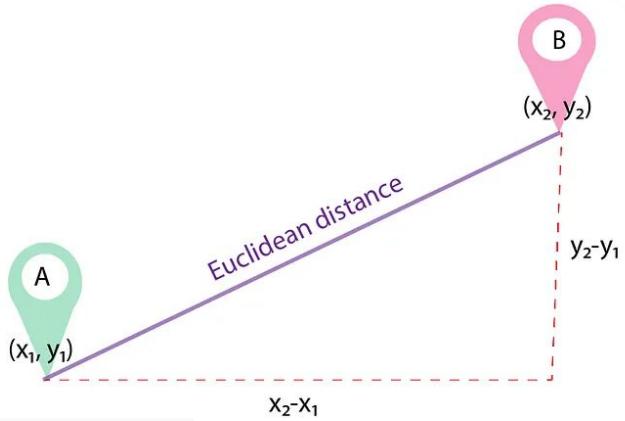
- Instance Memorization
- Similarity Measures/Distance Metrics
    - Euclidean Distance
    - Manhattan Distance
    - Cosine Similarity
- Lazy Learning

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Euclidean Distance

Euclidean distance is the straight line distance between 2 data points in a plane.

$$d(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$




```
def euclidean_distance_basic(point1, point2):

    if len(point1) != len(point2):
        raise ValueError("Points must have the same number of dimensions.")

    sum_sq_diff = 0
    for i in range(len(point1)):
        sum_sq_diff += (point1[i] - point2[i])**2
    return math.sqrt(sum_sq_diff)
```

```
p1 = (1, 2)
p2 = (4, 6)
distance = euclidean_distance_basic(p1, p2)
print(f"Euclidean distance (basic): {distance}")
```

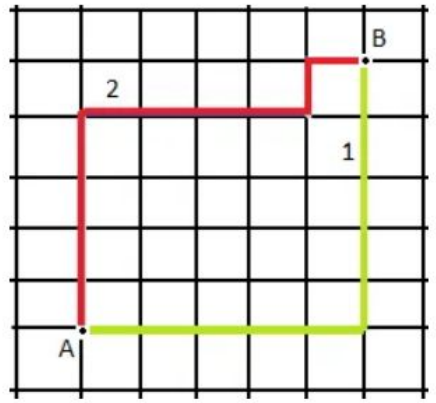
**Using Library Function**

```
from scipy.spatial import distance
euclidean_distance = distance.euclidean(p1, p2)
```

# Manhattan Distance

Also known as city block distance, or taxicab geometry if we need to calculate the distance between two data points in a grid-like path.

$$d = \sum_{i=1}^{n} |x_i - y_i|$$




```
def manhattan_distance(point1, point2):
    """Calculates the Manhattan distance between two points."""
    if len(point1) != len(point2):
        raise ValueError("Points must have the same number of dimensions.")

    distance = 0
    for i in range(len(point1)):
        distance += abs(point1[i] - point2[i])
    return distance
```

```
# Example usage:
dist = manhattan_distance(p1, p2)
print(f"Manhattan distance: {dist}")
```

**Using Library Function**

```
from scipy.spatial import distance
manhattan_distance = distance.cityblock(p1, p2)
```

# Industry Relevance

- **Instance Memorization -** In a KNN classification model for identifying flowers based on petal length and width, the model would store all labeled flower instances. When a new flower's petal dimensions are given, the model compares these dimensions with all stored flowers, finding the closest match to determine the likely species.
- **Similarity Measures:** In a recommender system that suggests movies to users, the system could use cosine similarity to measure the similarity between a user's movie-watching history and other users. By finding users with similar viewing habits, the model recommends movies that similar users have liked.
- **Lazy Learning** : In a customer support system using case-based reasoning, the system doesn't process or generalize knowledge until it receives a new customer inquiry. When a query arrives, it searches its case database for similar past cases to provide an answer.

# Advantages of Instance Based Learning

- Adaptability to new Data. (Example Fraud Detection)
- Intuitive and explainable predictions .(Example Medical Diagnosis)
- No need for extensive training time.( Example E-commerce recommendation System)
- Robustness to data diversity (Example Handwriting recognition)

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Comparison with Model Based Learning

- **Instance-Based Learning**: Stores and uses individual instances for predictions, has no separate training phase, is highly interpretable, and is ideal for tasks with local patterns or where data evolves frequently.

- **Model-Based Learning**: Uses a generalized model derived from the dataset, requires a training phase, and is suited for tasks needing high-level pattern recognition or scalability with large datasets.

For example, in fraud detection, an instance-based model can quickly adapt to new fraud cases, while a model-based approach might provide more comprehensive insights by recognizing broader fraud patterns in the data.

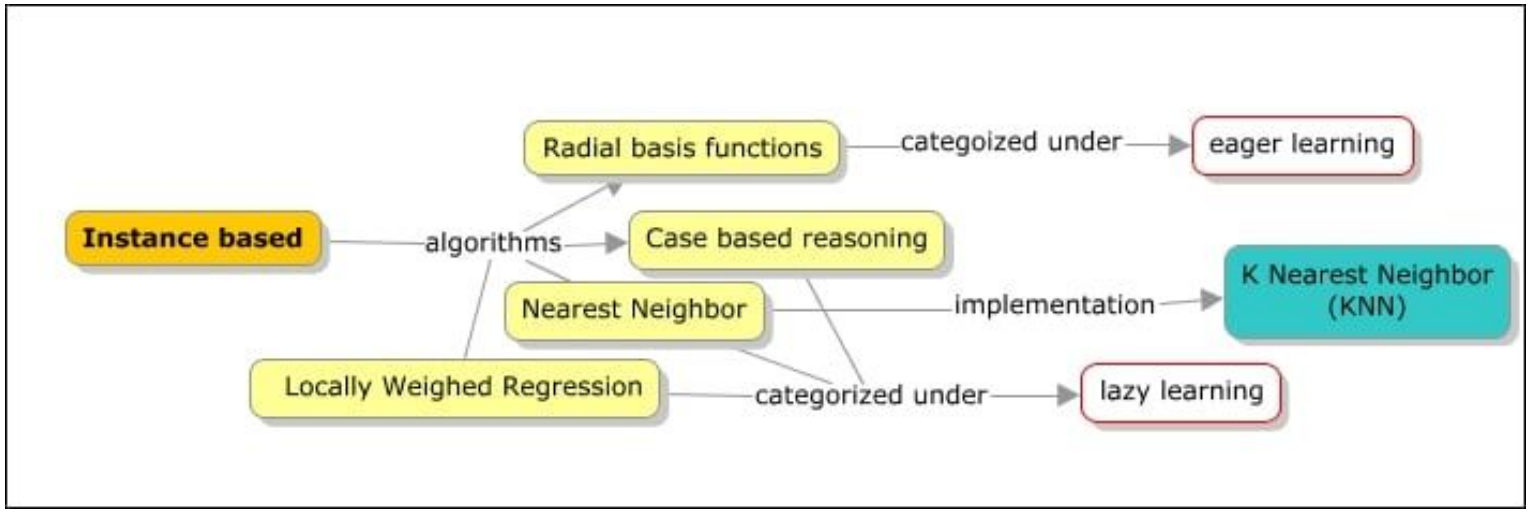
*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Instance-Based Learning (IBL) Vs Model Based Learning

| Aspect | Instance-Based Learning | Model-Based Learning |
|---|---|---|
| **Concept** | Learns by memorizing training examples and comparing new data to them. | Learns by building a mathematical model that generalizes from the data. |
| **Learning Approach** | "Remember and compare" | "Understand and predict" |
| **Examples** | k-Nearest Neighbors (k-NN), Case-Based Reasoning | Linear Regression, Decision Tree, Neural Networks |
| **Training Phase** | Simple - just store the data. | Involves model construction and parameter optimization. |
| **Prediction Phase** | Slow - compares new input to many stored instances. | Fast - directly uses the trained model for prediction. |
| **Computation Cost** | Low during training, high during prediction. | High during training, low during prediction. |
| **Memory Requirement** | High - needs to store all (or most) data. | Low - stores only model parameters. |
| **Generalization** | Local (depends on nearby examples). | Global (depends on learned model). |
| **Advantages** | Simple, flexible, no strong assumptions about data. | Compact, efficient predictions, can generalize well. |
| **Disadvantages** | Slow for large datasets, sensitive to noise. | Can underfit or overfit depending on model complexity. |

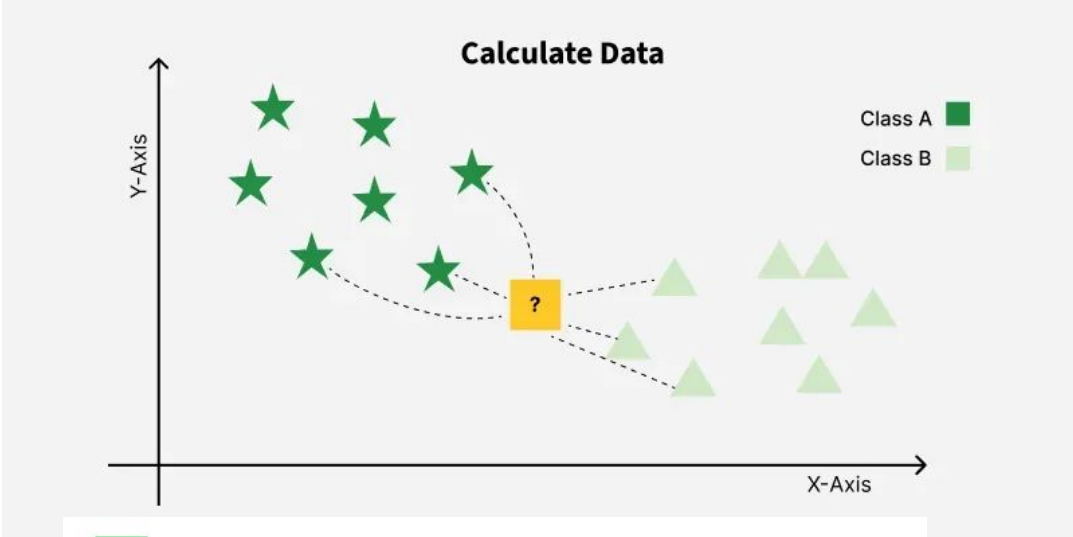
# Popular Algorithms for Instance Based Learning

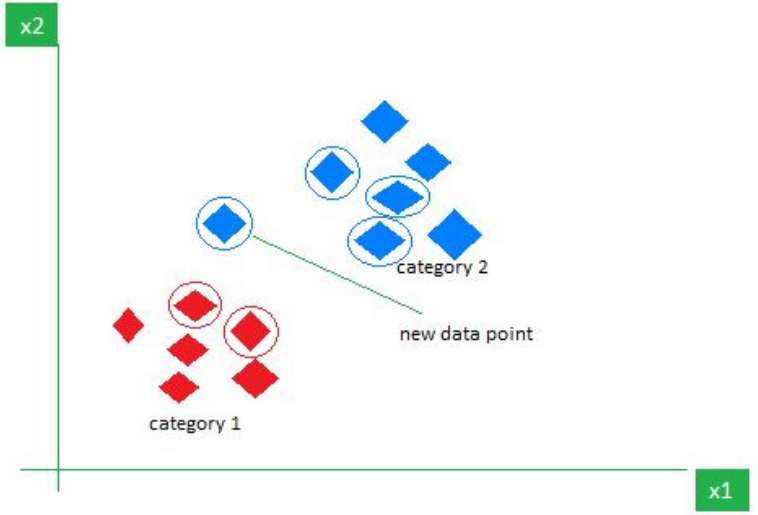
- k-Nearest neighbors (kNN)
- Locally Weighted Regression (LWR)
- Case Based Reasoning (CBR)
- Radial Basis Function Networks (RBFNs)




*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# k-Nearest neighbors (kNN)


*htm*

# Working of KNN algorithm

**Step 1: Selecting the optimal value of K**

K represents the number of nearest neighbors that needs to be considered while making prediction.

**Step 2: Calculating distance**

To measure the similarity between target and training data points Euclidean distance is used. Distance is calculated between data points in the dataset and target point.

**Step 3: Finding Nearest Neighbors**

The k data points with the smallest distances to the target point are nearest neighbors.

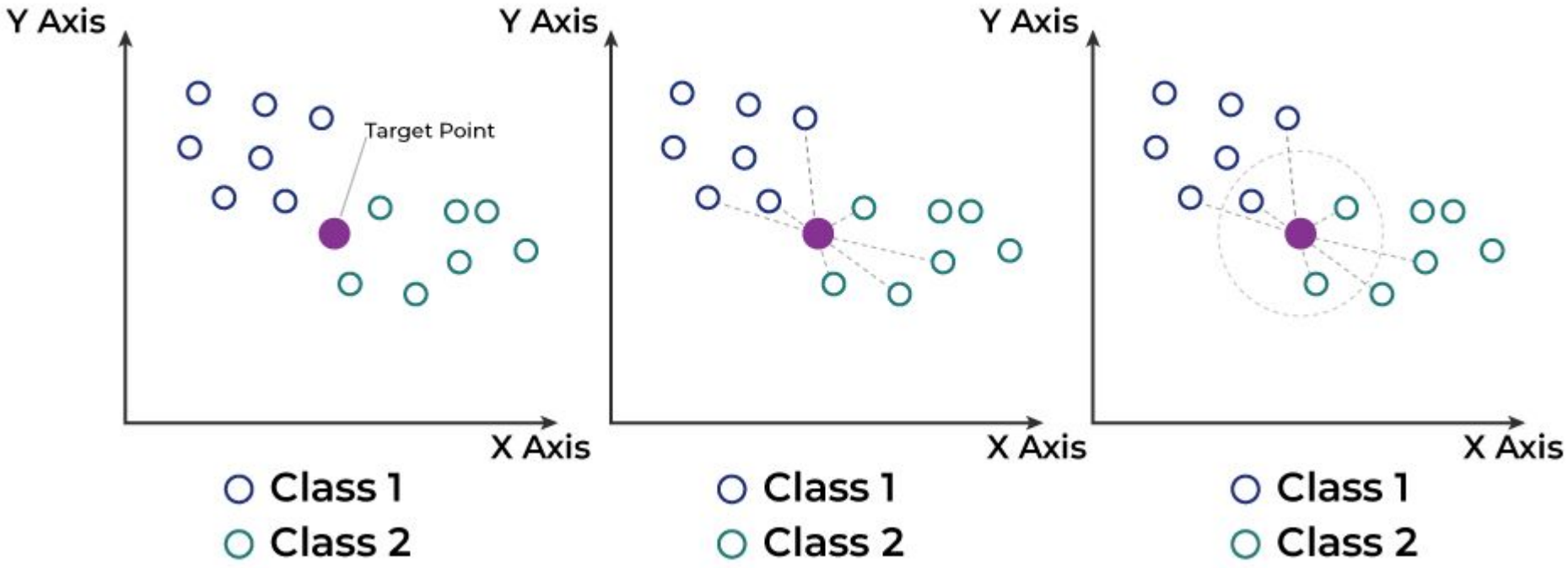
**Step 4: Voting for Classification or Taking Average for Regression**

When you want to classify a data point into a category like spam or not spam, the KNN algorithm looks at the K closest points in the dataset - majority voting.

In regression, the algorithm still looks for the K closest points. But instead of voting for a class in classification, it takes the average of the values of those K neighbors

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Working of KNN algorithm




*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# KNN from scratch

```
from collections import Counter

def knn(X_train, y_train, X_test, k=3):
    preds = []
    for test_point in X_test:
        # Compute Euclidean distance between test_point and every training sample
        distances = [np.linalg.norm(test_point - x) for x in X_train]
        # Get indices of the k smallest distances (i.e., nearest neighbors)
        k_idx = np.argsort(distances)[:k]
        # Retrieve the labels of those k nearest neighbors
        k_labels = [y_train[i] for i in k_idx]
        # Find the most common label among the k neighbors and append as
prediction
        preds.append(Counter(k_labels).most_common(1)[0][0])
    return preds
```

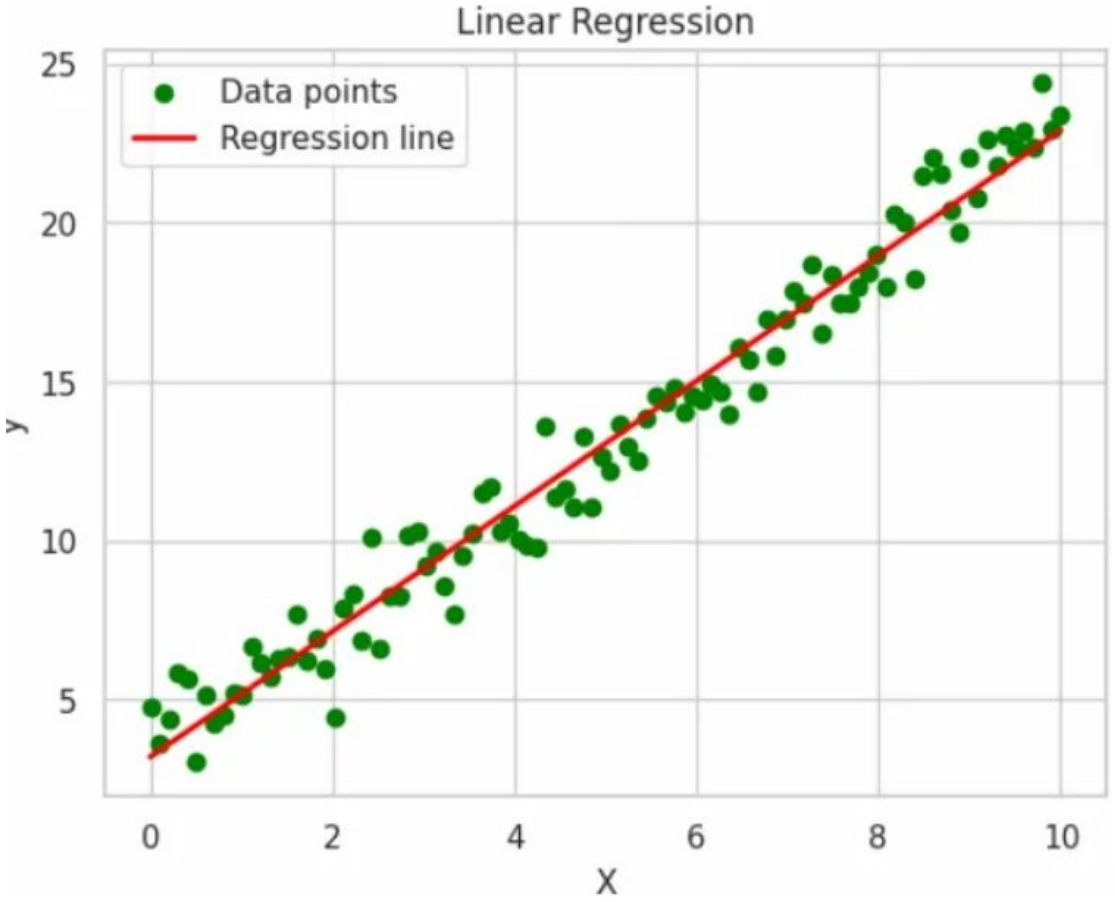
https://www.geeksforgeeks.org/machine-learning/ml-implementation-of-knn-classifier-using-sklearn/

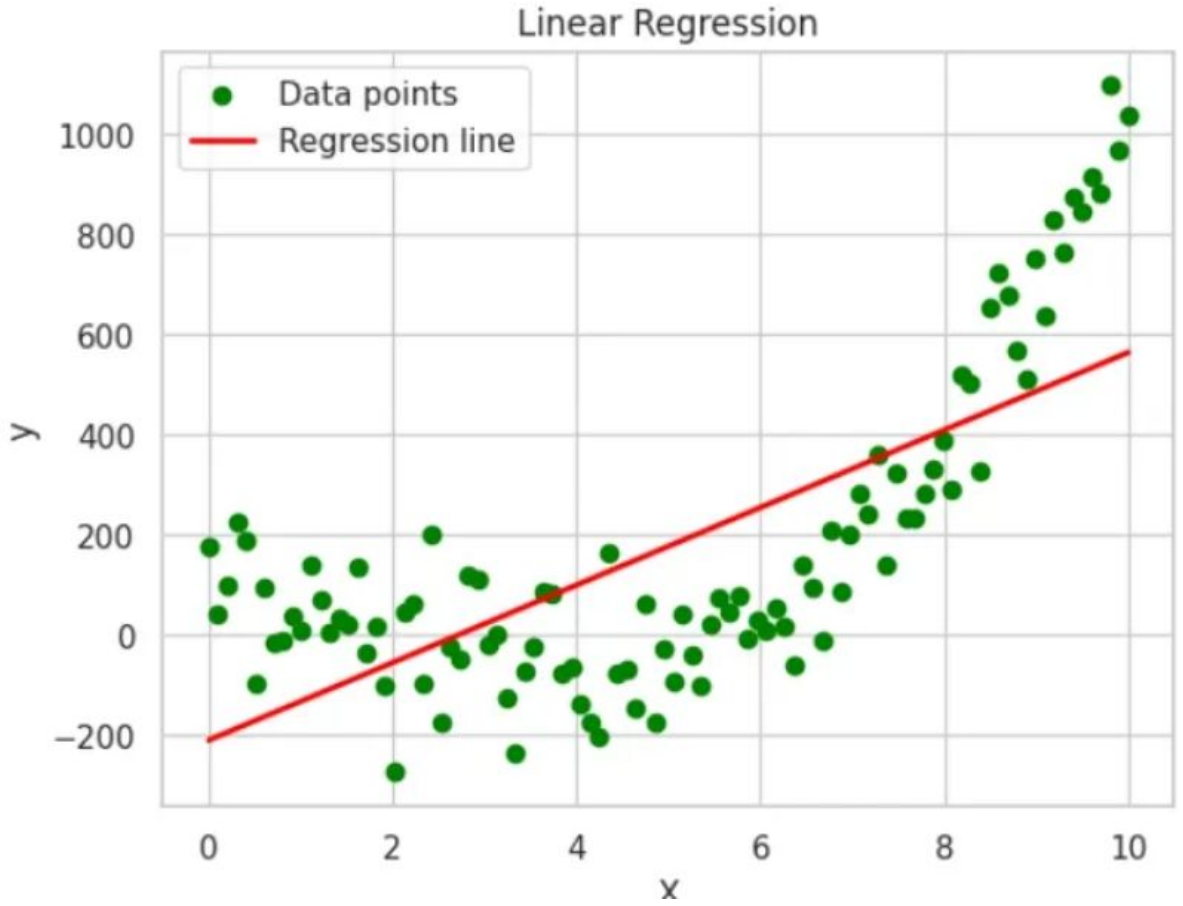
# KNN using Library

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score


knn_model = KNeighborsClassifier(n_neighbors=3)
knn_model.fit(X_train, y_train)
y_pred = knn_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

https://www.geeksforgeeks.org/machine-learning/ml-implementation-of-knn-classifier-using-sklearn/

# Locally weighted Linear Regression




*https://www.geeksforgeeks.org/machine-learning/ml-locally-weighted-linear-regression/*

# Locally weighted Linear Regression

- LWLR is a flexible method that adjusts the model to focus on the data points closest to each query point.
- Instead of creating one overall model like traditional linear regression, it creates a separate model for each prediction, using only the nearby data.
- This makes it useful when data behaves differently in different parts.
- The cost function is adjusted as : $\text{Minimize} \sum_i w_i(y_i - (\theta_0 + \theta_1 x_i))^2$
  - $w_i$: Weight for data point iii.
  - $y_i$: Actual value for data point iii.
  - $\theta_0, \theta_1$: Regression coefficients.

**Weight Calculation** $w_i = e^{-\frac{(x - x_i)^2}{2\tau^2}}$

- Here, $w_i$ is the weight for data point $x_i$, and τ\tauτ (bandwidth) controls the rate of weight decay.
- **Bandwidth (Tau)**: A critical parameter that governs how localized the regression is:

# Working of LWLR algorithm

1. **Select a Kernel Function:**

   A kernel function determines the weights assigned to the data points based on their distance from the query point. Common choices include the Gaussian kernel and the tricube kernel.

1. **Compute Weights:** For each query point, compute the weights for all data points using the chosen kernel function.
2. **Fit a Local Model:** Use weighted linear regression to fit a local model around the query point.
3. **Predict the Output:** Use the local model to predict the output for the query point.

# Practice LWLR algorithm

https://www.geeksforgeeks.org/machine-learning/ml-implementation-of-knn-classifier-using-sklearn/

# Radial Basis Function

A **Radial Basis Function (RBF)** is a type of real-valued function whose value depends only on the distance from a center point. It is commonly used in **machine learning**, particularly in:

- Function approximation ,Classification, Time-series prediction
- Kernel methods like Support Vector Machines (SVMs)
- Radial Basis Function Networks (a type of neural network)
- General form $\phi(x) = \phi(\|x - c\|)$

  - $x$: input point
  - $c$: center
  - $\|x - c\|$: distance between $x$ and $c$, usually Euclidean
  - $\phi$: the radial basis function (e.g., Gaussian)

# Radial Basis Function Network

An RBF Network (RBFN) is a type of artificial neural network that uses radial basis functions as activation functions. It is typically used for function approximation, pattern classification, and time series prediction.

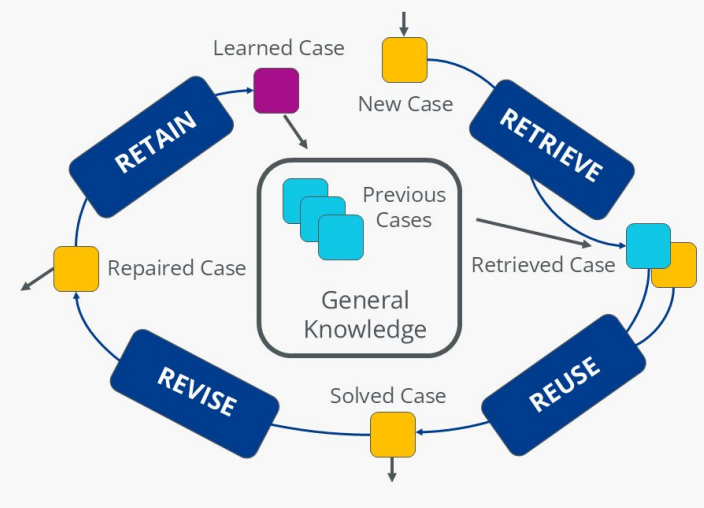
# Case-Based Reasoning

- This approach uses past experiences to solve new but similar problems, relying on analogy.
- **Case:** A case is a problem that has been previously solved and stored in the database.
- **Similarity:** The similarity measure is used to determine the degree of resemblance between past cases and the current situation.
- **Adaptation:** Adaptation is the process of modifying a retrieved past case to fit the current situation.

# Case-Based Reasoning Working

Case-based reasoning is a process that usually consists of four steps:

1. **Retrieve:** The CBR system examines the case database to find experiences most similar to the current problem based on its description.
2. **Reuse:** As a problem-solving approach, the solution of the case that is most similar to the description of the problem is used first. The first approach serves as the starting point for working on the new problem.
3. **Revise:** The proposed solution is tested in the new context and refined as needed to fit specific conditions. Adjustments or corrections are made if required.
4. **Retain:** The new problem-solving method is added to the case base for future requests. This creates an incremental learning process that ensures that the performance of the process increases with every case solved.



# Practical Considerations for Optimizing Instance-Based Learning

- Memory Management
- Computational Efficiency
- Handling Noisy data
- Feature Scaling and Selection
- Adaptation to Evolving Data

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Real World Applications of Instance-Based Learning

- Medical Diagnosis and Health care
- Fraud Detection in Financial Services
- Personalized Recommendations in E-commerce
- Image and Pattern Recognition
- Customer Support and Case Based Reasoning

*https://web.deu.edu.tr/doc/oreily/java/langref/ch10_js.htm*

# Bayesian learning

- Bayesian learning in machine learning is a probabilistic approach that uses Bayes' theorem to update beliefs about hypotheses based on observed data. `
- It combines prior knowledge with new evidence to refine predictions, making it suitable for complex or incomplete datasets.
- This approach provides a more nuanced understanding of uncertainty compared to traditional methods by working with probability distributions over model parameters and predictions

# Bayes' Theorem

- Bayesian view: All uncertain quantities (e.g. hypotheses) have probability distributions.
- Concept learning: Given data D, choose hypothesis h that maximizes posterior $P(h|D)$
- The theorem can be mathematically expressed as:

$$P(h \mid D) = \frac{P(D \mid h)\,P(h)}{P(D)}$$

- $P(h)$: prior belief in hypothesis
- $P(D \mid h)$: likelihood of data under $h$
- $P(D) = \sum_{h'} P(D \mid h')P(h')$: evidence (normalizing constant)

# Bayes' Theorem

- Suppose the weather of the day is cloudy. Now, you need to know whether it would rain today, given the cloudiness of the day. Therefore, you are supposed to calculate the probability of rainfall, given the evidence of cloudiness.
- That is, **P(Rain | Clouds)**, where finding whether it would rain today is the **Hypothesis (H)** and Cloudiness is the **Evidence (E)**. This is the posterior probability part of our equation.
- we know that 60% of the time, rainfall is caused by cloudy weather. Therefore, we have the probability of it being cloudy, given the rain, that is **P(clouds | rain) = P(E | H).**
- This is the backward probability where, E is the evidence of observing clouds given the probability of the rainfall, which is originally our hypothesis. Now, out of all the days, 75% of the days in a month are cloudy. This is the probability of cloudiness or **P(clouds)**.
- Also, since this is a rainy month of the year, it rains usually for 15 days out of 30 days. That is, the probability of hypothesis of rainfall or **P(H) is P(Rain)** = 15/30 = 0.5 or 50%. Now, let us calculate the probability of it raining, given the cloudy weather.

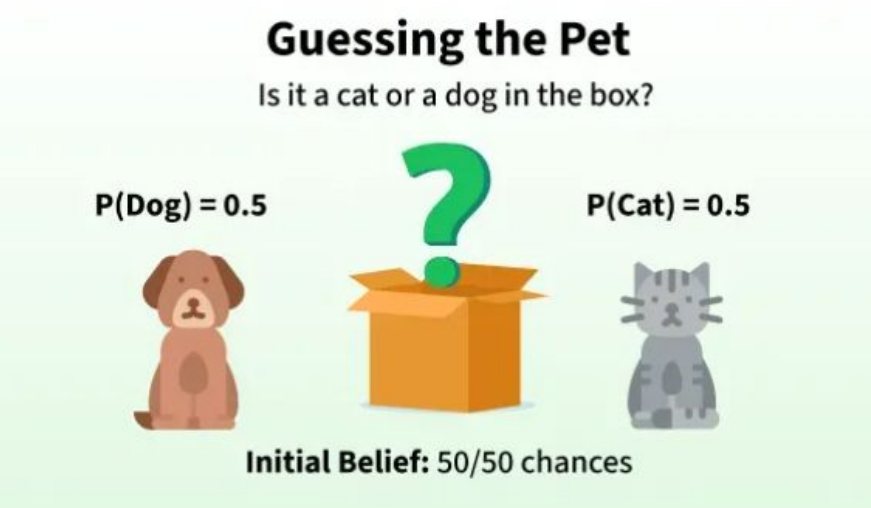
# Bayes' Theorem

$$P(Rain \mid Cloud) = (P(Cloud \mid Rain) * P(Rain)) / (P(Cloud))$$

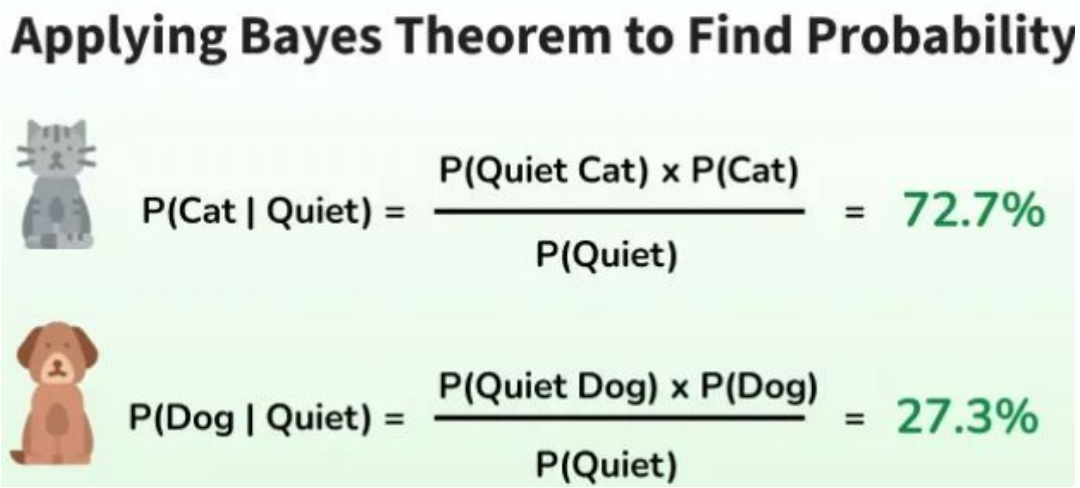
$$= (0.6 * 0.5) / (0.75)$$

$$= 0.4$$

# Bayes' Theorem







# Maximum Likelihood & Least-Squares Error

- Maximum likelihood estimation (MLE) chooses parameters θ maximizing **$L(\theta)=P(D|\theta)$**
- Least-squares arises when you assume Gaussian noise: MLE ≡ minimizing

# Maximum Likelihood & Least-Squares Error

- Maximum likelihood estimation (MLE) chooses parameters θ maximizing **$L(\theta)=P(D|\theta)$**
- Least-squares arises when you assume Gaussian noise: MLE ≡ minimizing

# Naïve bayes classifier

- Naïve Bayes is a probabilistic classifier based on Bayes' Theorem and the naïve assumption that features (symptoms) are conditionally independent given the class (disease).
- We want to predict whether a patient has a Cold (Yes/No) based on symptoms:
  - Fever
  - Cough
  - Fatigue
- Bayes' Theorem Recap: $$P(Y \mid X_1, X_2, ..., X_n) = \frac{P(Y) \cdot \prod_{i=1}^{n} P(X_i \mid Y)}{P(X_1, X_2, ..., X_n)}$$
- In our case: $$P(\text{Cold} \mid \text{Fever}, \text{Cough}) \propto P(\text{Cold}) \cdot P(\text{Fever} \mid \text{Cold}) \cdot P(\text{Cough} \mid \text{Cold})$$

# Cold Diagnosis- Example Dataset (Simplified)

| Cold | Fever | Cough | Fatigue |
|---|---|---|---|
| Yes | Yes | Yes | Yes |
| Yes | Yes | Yes | No |
| Yes | No | Yes | Yes |
| No | Yes | No | Yes |
| No | No | Yes | No |
| No | No | No | No |

Step 1: Prior Probability

$$P(\text{Cold=Yes}) = \frac{3}{6} = 0.5, \quad P(\text{Cold=No}) = 0.5$$

Step 2: Likelihood

**When Cold = Yes:**

- $P(\text{Fever=Yes} \mid \text{Cold=Yes}) = \frac{2}{3}$
- $P(\text{Cough=Yes} \mid \text{Cold=Yes}) = 1.0$

**When Cold = No:**

- $P(\text{Fever=Yes} \mid \text{Cold=No}) = \frac{1}{3}$
- $P(\text{Cough=Yes} \mid \text{Cold=No}) = \frac{1}{3}$

Step 3: Apply Naïve Bayes

$$P(\text{Cold=Yes} \mid \text{Fever=Yes, Cough=Yes}) \propto 0.5 \cdot \frac{2}{3} \cdot 1 = 0.333$$

$$P(\text{Cold=No} \mid \text{Fever=Yes, Cough=Yes}) \propto 0.5 \cdot \frac{1}{3} \cdot \frac{1}{3} = 0.055$$

# Step-by-Step Naïve Bayes Classification

Step 4: Normalize

$$\text{Total} = 0.333 + 0.055 = 0.388$$

$$P(\text{Cold=Yes} \mid \text{Fever=Yes, Cough=Yes}) = \frac{0.333}{0.388} \approx 0.858$$

$$P(\text{Cold=No} \mid \text{Fever=Yes, Cough=Yes}) = \frac{0.055}{0.388} \approx 0.142$$

A patient with **Fever and Cough** has an **85.8% probability of having a Cold**, based on the Naïve Bayes model.

# Bayesian Belief Networks

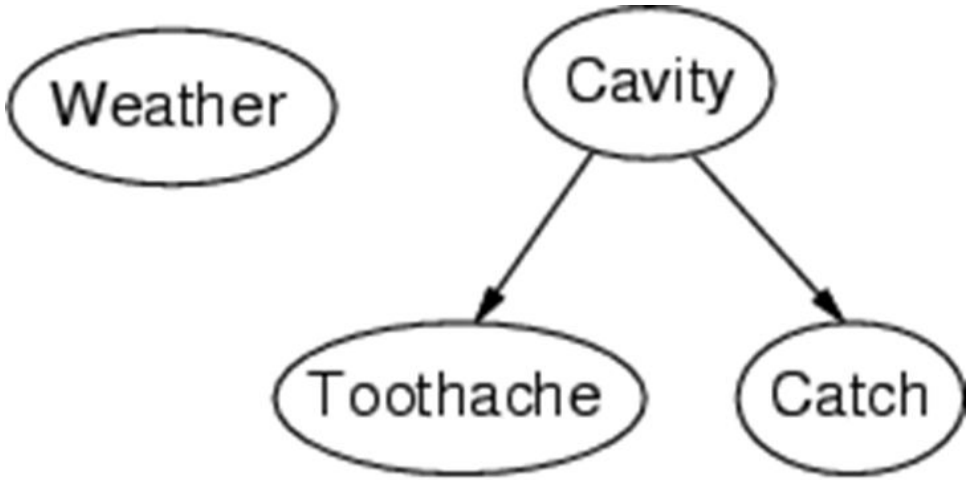
- A simple, graphical notation for conditional independence assertions and hence for compact specification of full joint distributions

- Syntax:
  - a set of nodes, one per variable
  - a directed, acyclic graph (link ≈ "directly influences")
  - a conditional distribution for each node given its parents:
  - P (Xi | Parents (Xi))

- In the simplest case, conditional distribution represented as a conditional probability table (CPT) giving the distribution over Xi for each combination of parent values
  A node is independent of its nondescendents given its parents.

# Bayesian Belief Networks - Example

- Topology of network encodes conditional independence assertions:



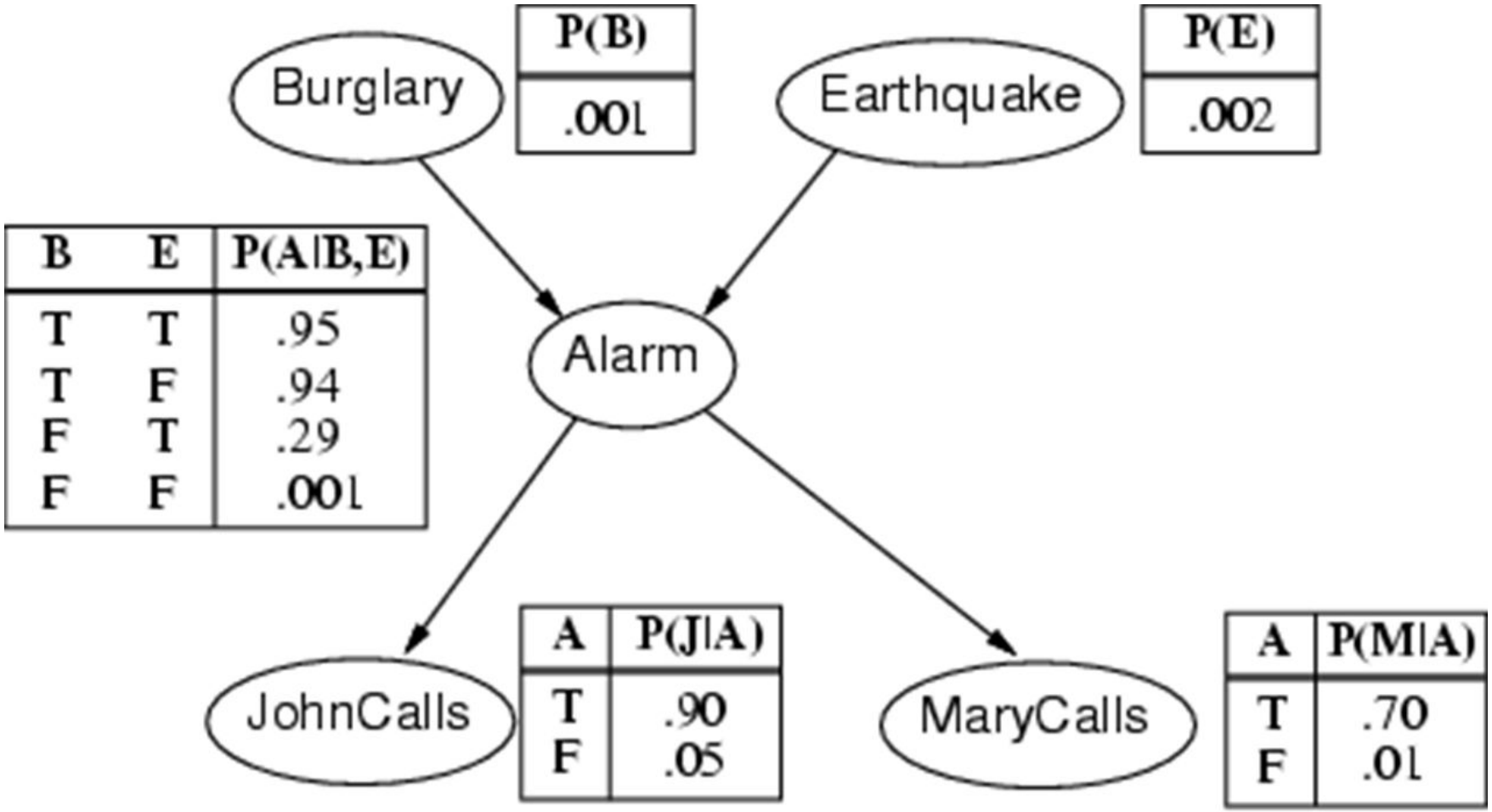

- Weather is independent of the other variables
- Toothache and Catch are conditionally independent given Cavity

# Bayesian Belief Networks - Real World Example

- I'm at work, neighbor John calls to say my alarm is ringing, but neighbor Mary doesn't call. Sometimes it's set off by minor earthquakes. Is there a burglar?
- Variables: Burglary, Earthquake, Alarm, JohnCalls, MaryCalls

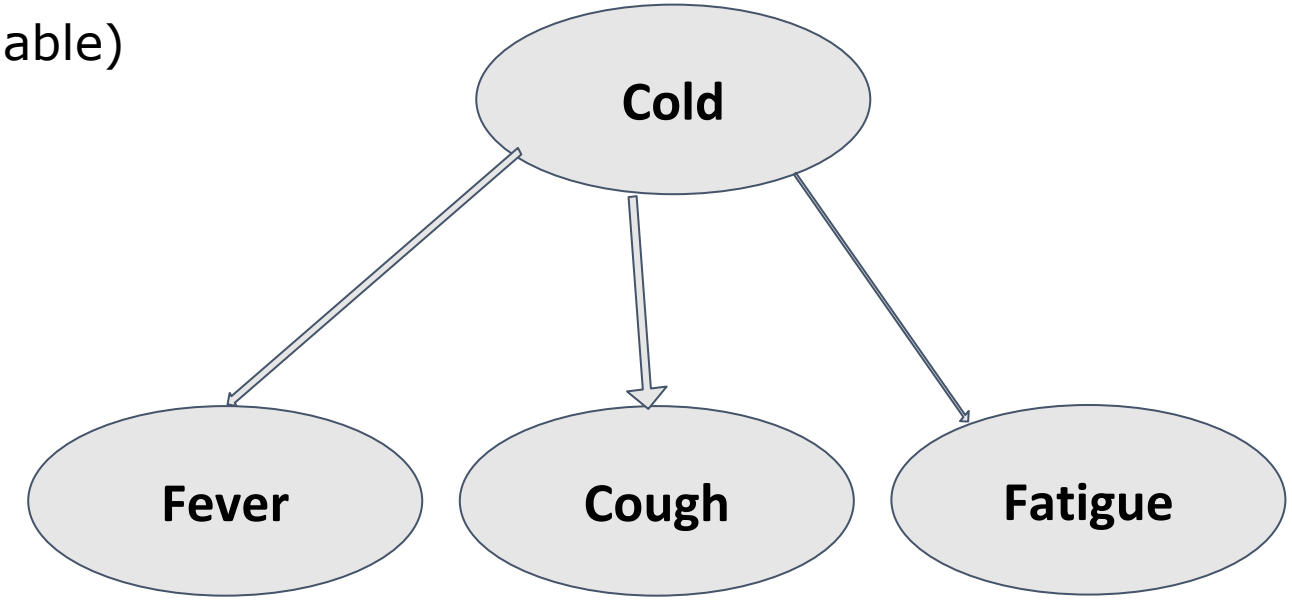
- Network topology reflects "causal" knowledge:
  - A burglar can set the alarm off
  - An earthquake can set the alarm off
  - The alarm can cause Mary to call
  - The alarm can cause John to call

# Bayesian Belief Networks - Real World Example



# Cold Diagnosis using Bayesian Network

- Variables in the Network:
  - Cold (Yes/No) - Disease (latent variable)
  - Cough (Yes/No) - Symptom
  - Fever (Yes/No) - Symptom
  - Fatigue (Yes/No) - Symptom




- The Cold node influences the presence of symptoms like Fever, Cough, and Fatigue.

# Cold Diagnosis- Example Dataset (Simplified)

| Cold | Fever | Cough | Fatigue |
|---|---|---|---|
| Yes | Yes | Yes | Yes |
| Yes | Yes | Yes | No |
| Yes | No | Yes | Yes |
| No | Yes | No | Yes |
| No | No | Yes | No |
| No | No | No | No |

P(Cold=Yes) = 3/6 = 0.5
P(Cold=No) = 3/6 = 0.5

P(Fever=Yes | Cold=Yes) = 2/3
P(Fever=No | Cold=Yes) = 1/3

P(Fever=Yes | Cold=No) = 1/3
P(Fever=No | Cold=No) = 2/3

P(Cough=Yes | Cold=Yes) = 1.0
P(Cough=Yes | Cold=No) = 1/3

P(Fatigue=Yes | Cold=Yes) = 2/3
P(Fatigue=Yes | Cold=No) = 1/3

# Cold Diagnosis - Example Inference

**Q: What is the probability that a patient has a cold given that they have Fever and Cough?**

We want to compute:

P(Cold=Yes∣Fever=Yes,Cough=Yes)P(Cold=Yes \mid Fever=Yes, Cough=Yes)
P(Cold=Yes∣Fever=Yes,Cough=Yes)

Using **Bayes' Theorem** and assuming conditional independence between symptoms given the disease:

$$P(Cold = Yes \mid Fever, Cough) \propto P(Cold = Yes) \cdot P(Fever \mid Cold = Yes) \cdot P(Cough \mid Cold = Yes)$$

Substitute the values:

$$P \propto 0.5 \cdot 2/3 \cdot 1.0 = 0.333$$

Similarly, for Cold=No:

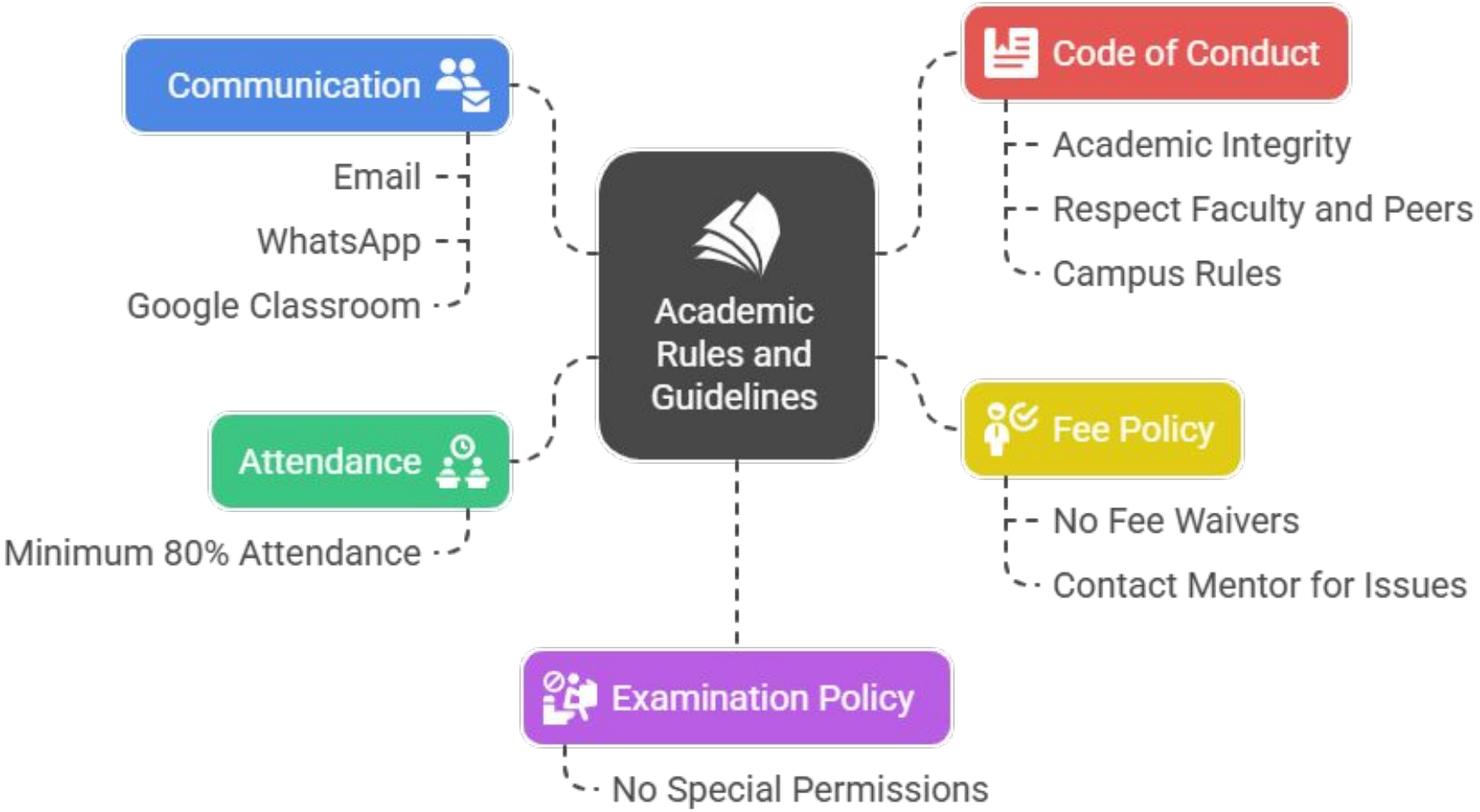
$$P \propto 0.5 \cdot 1/3 \cdot 1/3 = 0.055$$

Normalize:

$$P(Cold = Yes \mid Fever, Cough) = \frac{0.333}{0.333 + 0.055} \approx 0.858$$

There is approximately an **85.8% chance** that the patient has a **cold** if they have **both fever and cough**

# GSFCU & Values : Academic Rules and Guidelines for A.Y 2025-26



# Thank You..